

Image Recognition



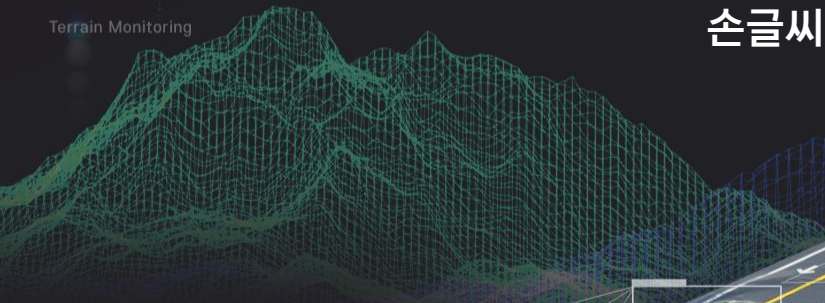
강원혁신플랫폼

컴퓨터 비전



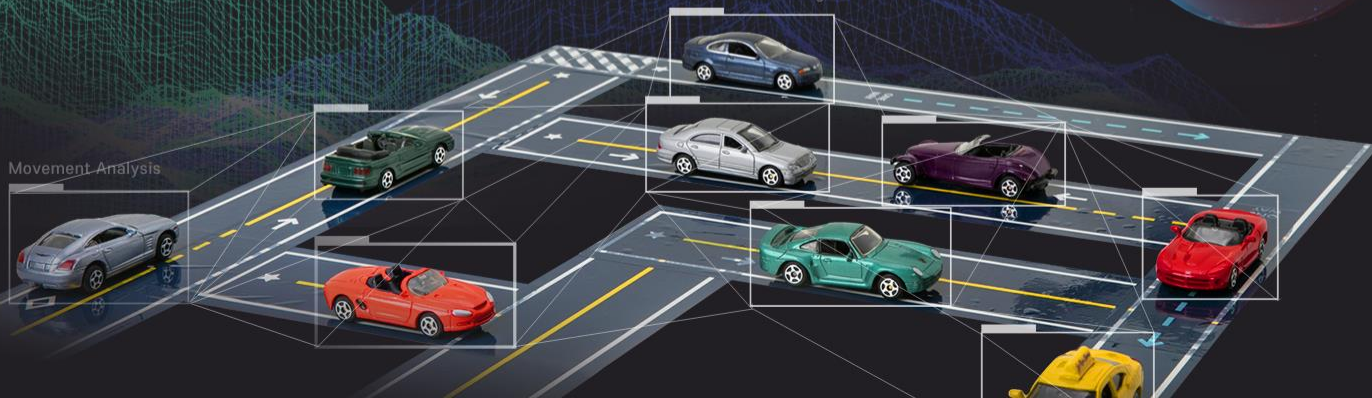
손글씨 숫자 작성과 추론 실습

Terrain Monitoring



Autonomous Driving

Movement Analysis



MNIST 데이터셋을 이용하여 0~9까지의 숫자를 분류하는
Multi Classification 모델을 만들고 이 모델을 이용해
우리가 직접 손으로 작성한 숫자 이미지를 분류 예측한 결과를 확인해보자

이것은 다중 이미지 분류(Image Multi Classification)의 문제임

MNIST 손글씨 숫자 데이터를 이용한
10가지 이미지 분류 전체 과정 실습



인공신경망(딥러닝)의
세부 알고리즘 학습

10가지 클래스 인덱스

“0”:0 “1”:1 “2”:2 “3”:3 “4”:4
“5”:5 “6”:6 “7”:7 “8”:8 “9”:9

※ 실습은 구글 Colab을 이용함



손으로 작성한 숫자 이미지를 모델에 넣어 분류 예측 수행하기

- ▶ 내 구글 드라이브에 연동
- ▶ 구글 Colab 노트북 화면에서 실행 버튼 클릭



```
# 내 구글 드라이브에 연동  
from google.colab import drive
```

```
drive.mount('/content/gdrive')
```



```
Mounted at /content/gdrive
```

손으로 작성한 숫자 이미지를 모델에 넣어 분류 예측 수행하기

➡ [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴 선�ک

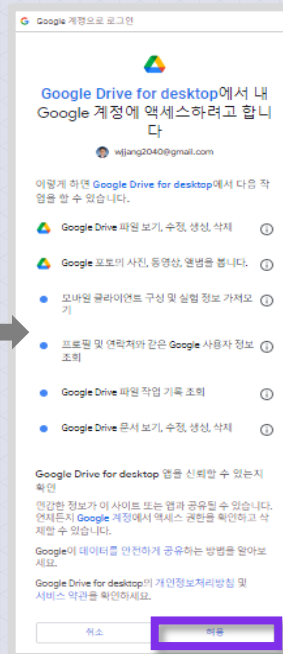
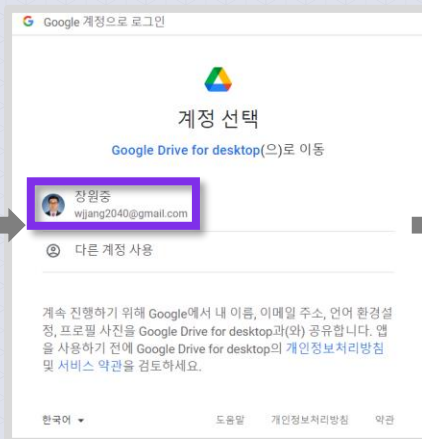
참고 아래와 같이 Google Drive에 연동

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니요

Google Drive에 연결



손으로 작성한 숫자 이미지를 모델에 넣어 분류 예측 수행하기

- ➡ 필요한 패키지와 모듈 불러오기
- ➡ 구글 Colab 노트북 화면에서 실행 버튼 클릭



필요한 패키지와 모듈을 불러옴

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

손으로 작성한 숫자 이미지를 모델에 넣어 분류 예측 수행하기

- 구글 Colab 노트북 화면에서 실행 버튼 클릭

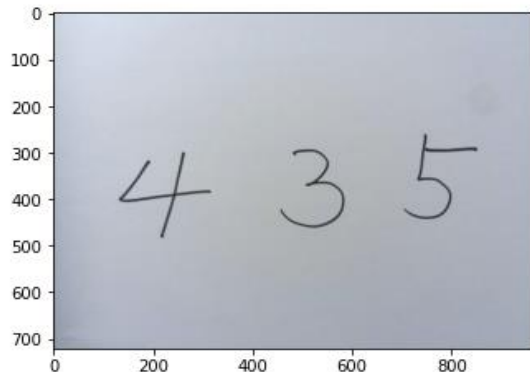
▶ # 숫자가 쓰여져 있는 이미지 파일을 읽어 온다.

```
frame = 'gdrive/My Drive/CV/Digits Classify/images/435.jpg'
```

```
img = cv2.imread(frame)
print("형상:", img.shape)
```

```
plt.imshow(img);
print("width: {} pixels".format(img.shape[1]))
print("height: {} pixels".format(img.shape[0]))
print("channels: {}".format(img.shape[2]))
```

▶ 형상: (721, 960, 3)
width: 960 pixels
height: 721 pixels
channels: 3



손으로 작성한 숫자 이미지를 모델에 넣어 분류 예측 수행하기

- 이미지 분석을 위해 HSV Filter를 적용하고, 채널(Channel)를 분리해서 명도(Value) 이미지를 사용함
- 구글 Colab 노트북 화면에서 실행 버튼 클릭



이미지를 BGR to HSV(HSV filter 적용)

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

hue, saturation, value = cv2.split(hsv)

plt.imshow(value)

색상(hue), 채도(saturation), 명도(value)

명도 이미지를 보여준다.

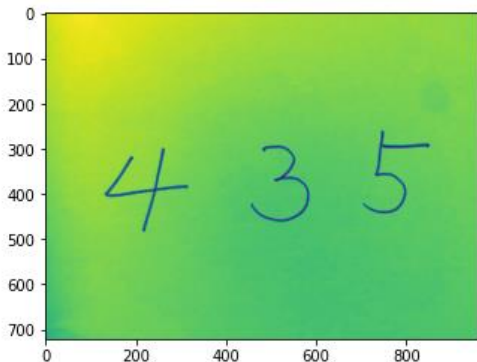
BGR



HSV 변경



<matplotlib.image.AxesImage at 0x7fa6724f0c50>



모폴로지(Morphology) 연산

- 모폴로지(Morphology) 연산(침식, 팽창, 열림, 닫힘, 그레디언트, 탐헛, 블랫헛)을 사용하기 위해 커널(Kernel)을 정의한다.
- 모폴로지는 영상 분야에서 노이즈 제거, 구멍 채우기, 끊어진 선 이어 붙이기 등에 쓰이는 형태학적 연산을 말한다.
- 모폴로지 연산은 검은색과 흰색으로만 구성되어 있는 바이너리(Binary) 이미지에 적용할 수 있다.
- 모폴로지 연산으로는 침식(Erosion), 팽창(Dilatation), 열림(Opening), 닫힘(Closing)이 있다.



■ 모폴로지 연산을 사용하기 위해 구조화 요소 커널(Structuring element kernel)을 정의함

- 구조화 요소 커널은 1이 채워진 모양에 따라 사각형, 타원형, 십자형 등으로 사용할 수 있음
- 여기서는 사각형 모양을 사용함
- “1”이 채워진 사각형 모양의 구조화 요소 커널 생성



```
# morphological operations 을 사용하기 위해 kernel 정의  
# - 구조화 요소 커널, 직사각형(cv2.MORPH_RECT) (3x3) 생성
```

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))  
print(kernel.shape)  
print(kernel)
```



```
(3, 3)  
[[1 1 1]  
 [1 1 1]  
 [1 1 1]]
```

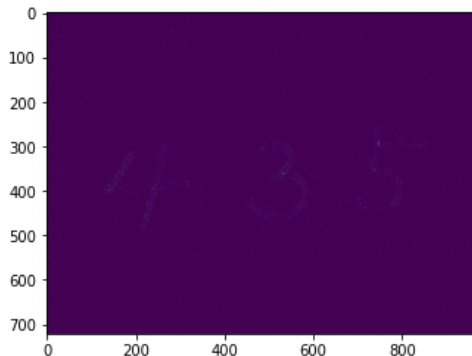
손으로 작성한 숫자 이미지를 모델에 넣어 분류 예측 수행하기

- 명도(Value)에 탑햇 연산(원본 - 열림 연산) 적용
- 탑햇 연산은 원본에서 열림 연산 적용 결과를 빼면 값이 크게 튀는 밝은 부분을 강조할 수 있음

```
# topHat operations 을 적용한다.
# - 탑햇 연산(cv2.MORPH_TOPHAT) : 원본 - 열림 연산
# - 즉 원본에서 열림 연산 적용 결과를 빼면 값이 크게 튀는 밝은 영역을 강조할 수 있다.

topHat = cv2.morphologyEx(value, cv2.MORPH_TOPHAT, kernel)
plt.imshow(topHat)
```

<matplotlib.image.AxesImage at 0x7f200af8d9d0>



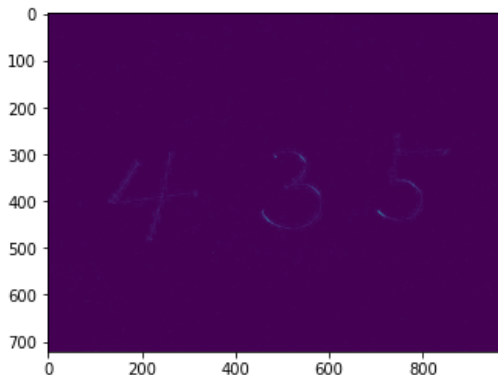
밝은 영역 강조

손으로 작성한 숫자 이미지를 모델에 넣어 분류 예측 수행하기

- 명도(Value)에 블랙햇 연산(닫힘 연산 - 원본) 적용
- 블랙햇 연산은 닫힘 연산 적용 결과에서 원본을 빼면 어두운 부분을 강조할 수 있음

```
# blackHat operations 을 적용한다.  
# - 블랙햇(cv2.MORPH_BLACKHAT) : 닫힘 연산 - 원본  
# - 즉 닫힘 연산 적용 결과에서 원본을 빼면 어두운 부분을 강조할 수 있다.  
blackHat = cv2.morphologyEx(value, cv2.MORPH_BLACKHAT, kernel)  
plt.imshow(blackHat)
```

<matplotlib.image.AxesImage at 0x7f200a437810>



어두운 영역 강조

손으로 작성한 숫자 이미지를 모델에 넣어 분류 예측 수행하기

🔄 명도(Value)에 모폴로지 연산 덧셈, 뺄셈을 수행함

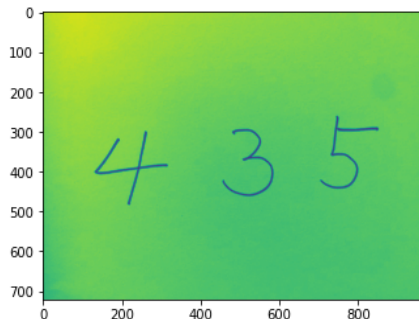
▶ # 모폴로지(Morphology)operations 간에 add and subtract

cv2.add() : 두 영상의 같은 위치에 존재하는 픽셀 값을 더하여 결과 영상의 픽셀 값으로 설정한다.
덧셈 결과가 255보다 크면 픽셀 값을 255로 설정하는 saturate 기능이 적용되어 있다.

cv2.subtract() : 두 영상의 같은 위치에 존재하는 픽셀 값에 대하여 뺄셈 연산을 수행하여 결과 영상의 픽셀 값으로 설정한다.
뺄셈 결과가 0보다 작으면 픽셀 값을 0으로 설정한다.

```
add = cv2.add(value, topHat)
subtract = cv2.subtract(add, blackHat)
plt.imshow(subtract)
```

🖼️ <matplotlib.image.AxesImage at 0x7fa671c1fb10>



add = 명도(Value) + 탑햇(TopHat)

subtract = add + 블랙햇(blackHat)

손으로 작성한 숫자 이미지를 모델에 넣어 분류 예측 수행하기

- 명도(Value)에 모폴로지 연산 덧셈, 뺄셈을 수행함

cv2.add()

두 영상의
같은 위치에 존재하는 픽셀 값을 더하여
영상의 픽셀 값을 설정



cv2.subtract()

두 영상의
같은 위치에 존재하는 픽셀 값에 대하여
뺄셈 연산을 수행하여
결과 영상의 픽셀 값으로 설정



가우시안 필터(Gaussian blur)

- 가우시안 필터(블러링)는 모든 픽셀에 똑같은 가중치를 부여했던 평균 블러링과 달리 중심에 있는 픽셀에 높은 가중치를 부여한다.
- 가우시안 필터는 전체적으로 밀도가 동일한 노이즈, 백색 노이즈를 제거하는데 효과적이다.

손으로 작성한 숫자 이미지를 모델에 넣어 분류 예측 수행하기

🔄 모폴로지 연산 간에 덧셈, 뺄셈을 수행한 이미지에 가우시안 필터(Gaussian blur)를 적용함

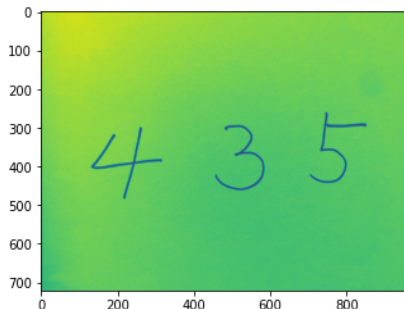
▶ # 모폴로지(Morphology)operations 간에 add and subtract

cv2.add() : 두 영상의 같은 위치에 존재하는 픽셀 값을 더하여 결과 영상의 픽셀 값으로 설정한다.
덧셈 결과가 255보다 크면 픽셀 값을 255로 설정하는 saturate 기능이 적용되어 있다.

cv2.subtract() : 두 영상의 같은 위치에 존재하는 픽셀 값에 대하여 뺄셈 연산을 수행하여 결과 영상의 픽셀 값으로 설정한다.
뺄셈 결과가 0보다 작으면 픽셀 값을 0으로 설정한다.

```
add = cv2.add(value, topHat)
subtract = cv2.subtract(add, blackHat)
plt.imshow(subtract)
```

🖼️ <matplotlib.image.AxesImage at 0x7fa671c1fb10>



손으로 작성한 숫자 이미지를 모델에 넣어 분류 예측 수행하기

➡ 가우시안 필터를 적용한 이미지에 임계 값(Threshold)을 적용함

```
1 # blur 이미지에 threshold를 적용한다.  
2  
3 # adaptive Threshold는 이미지의 구역구역마다 threshold를 실행 시켜주는 것이다.  
4 # cv2.adaptiveThreshold(img, value, adaptivemethod, thresholdType, blockSize, C)  
5 # - cv2.ADAPTIVE_THRESH_GAUSSIAN_C: X, Y를 중심으로 block Size * block Size 안에 있는 픽셀 값의 평균에서 C를 뺀 값을 문턱값으로 한다.  
6  
7 thresh = cv2.adaptiveThreshold(blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV, 19, 9)  
8 plt.imshow(thresh)
```

cv2.adaptiveThreshold() 함수

이미지의 구역구역마다 Threshold를 실행시킴

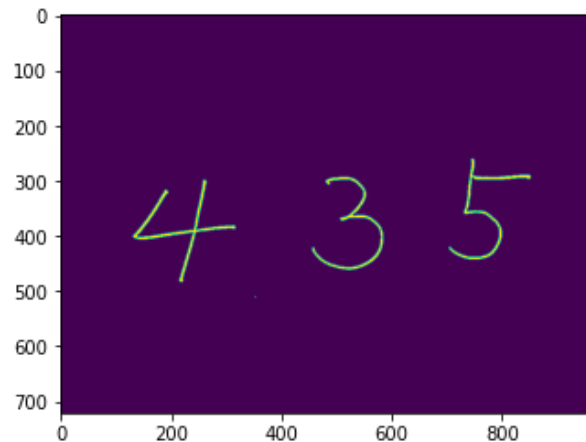
ADAPTIVE_THRESH_GAUSSIAN_C

X, Y를 중심으로 Block Size × Block Size 안에 있는
픽셀 값의 평균에서 C(여기서는 9)를 뺀 값을
문턱 값으로 함

손으로 작성한 숫자 이미지를 모델에 넣어 분류 예측 수행하기

➡ 가우시안 필터를 적용한 이미지에 임계 값(Threshold)을 적용함

```
1 # blur 이미지에 threshold를 적용한다.  
2  
3 # adaptive Threshold는 이미지의 구역구역마다 threshold를 실행 시켜주는 것이다.  
4 # cv2.adaptiveThreshold(img, value, adaptivemethod, thresholddtype, blockSize, C)  
5 # - cv2.ADAPTIVE_THRESH_GAUSSIAN_C: X, Y를 중심으로 block Size + block Size 안에 있는 픽셀 값의 평균에서 C를 뺀 값을 문턱값으로 한다.  
6  
7 thresh = cv2.adaptiveThreshold(blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV, 19, 9)  
8 plt.imshow(thresh)
```



손으로 작성한 숫자 이미지를 모델에 넣어 분류 예측 수행하기

➡ 적응형 Threshold(Adaptive threshold) 이미지에 윤곽선(Contour)을 찾을

`cv2.findContours()` 함수

- 1 영상이나 이미지의 윤곽선(컨투어)를 검출하기 위해 사용함
- 2 영상이나 이미지에서 외곽과 내곽의 윤곽선을 검출함
- 3 OpenCV 버전이 업그레이드 됨에 따라 파라미터 개수의 변동이 있다
(버전4 이상은 2개, 버전4 미만은 3개)

다음은 적응형 Threshold(Adaptive threshold) 이미지에 윤곽선(Contour)을 찾는 코드

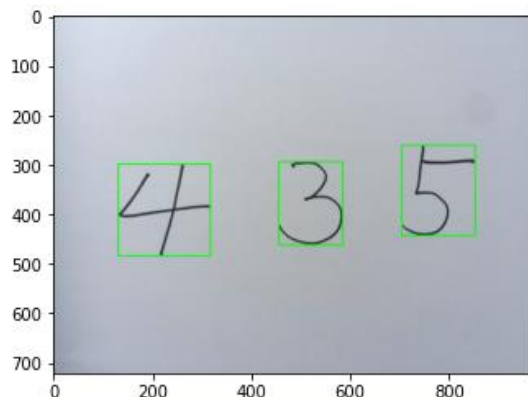
```
1 # cv2.findContours() 함수가 OpenCV 버전이 업그레이드 됨에 따라 파라미터 개수에 변동이 있다.
2 # - OpenCV 버전4 이상은 파라미터 개수가 2개, OpenCV 버전4 미만은 파라미터 개수가 3개
3
4 # cv2.findContours() : 영상이나 이미지의 윤곽선(컨투어)을 검출하기 위해 사용한다.
5 # 영상이나 이미지에서 외곽과 내곽의 윤곽선(컨투어)을 검출할 수 있다.
6
7 cv2MajorVersion = cv2.__version__.split(".")[0]
8 print('openCV version : ', cv2MajorVersion) # openCV version : 4
9
10 # thresh 이미지에서 contour들을 찾아온다.
11 if int(cv2MajorVersion) >= 4:
12     contours, hierarchy = cv2.findContours(thresh, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)
13 else:
14     imageContours, contours, hierarchy = cv2.findContours(thresh, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)
15
16 print("contours len : ", len(contours)) # contours len : 4
17 print("contours : ", contours[0]) # [[[352 510]] [[352 511]] [[353 511]] [[353 510]]]
```

- 적응형 Threshold(Adaptive threshold) 이미지에 윤곽선을 찾아 숫자 이미지 배열에 저장함
- 원본 이미지의 숫자 이미지 위치에 사각형으로 표시함

```

1  # 적응형 Threshold(adaptive threshold) 이미지에서 contour들을 찾아서 digit 이미지를 저장
2
3  img_digits = []          # digit 이미지가 저장되는 배열 정의
4  margin = 10             # digit 이미지 저장 여유 마진
5
6  for contour in contours:
7      x, y, w, h = cv2.boundingRect(contour)          # contour 주변을 사각형으로 찾아옴
8      print('x =', x, 'y =', y, 'w =', w, 'h =', h)
9
10     # 찾아온 이미지가 너무 작다면 무시함
11     if w * h < 100: continue
12
13     # Y and X [y-margin:y+h+margin, x-margin:x+w+margin]
14     # contour boxing 위치 좌표를 계산하여 thresh 이미지로부터 digit image 를 구해 img_digits 배열에 넣어준다.
15     img_digits.append(thresh[y-margin:y+h+margin, x-margin:x+w+margin])
16
17     # 원본 이미지에 contour boxing 영역을 사각형으로 표시해 준다.
18     cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
19     print(x, x+w, y, y+h) ; print()
20 plt.imshow(img)
    
```

첫 번째 숫자 이미지 위치에 사각형 표기

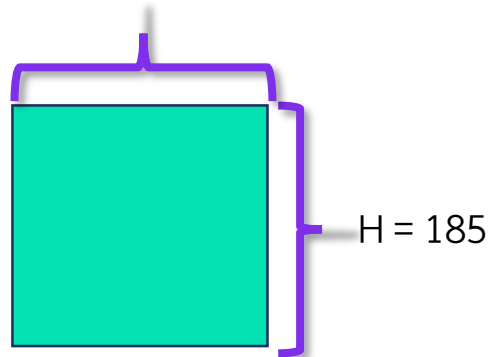


$x = 352$ $y = 510$ $w = 2$ $h = 2$
 $x = 131$ $y = 299$ $w = 186$ $h = 185$
 131 317 299 484

$x = 455$ $y = 294$ $w = 129$ $h = 168$
 455 584 294 462

$x = 703$ $y = 261$ $w = 148$ $h = 182$
 703 851 261 443

$W = 186$



$X = 131$ $Y = 299$

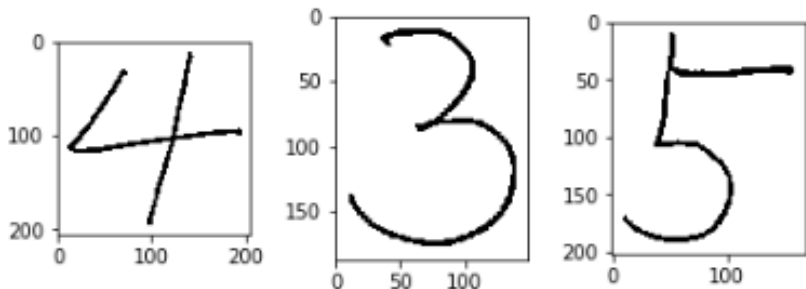
$X + W = 131 + 186 = 317$

$Y + H = 299 + 185 = 484$

- 숫자 이미지 배열(img_digits)에 들어 있는 숫자 이미지 확인
- 숫자 이미지 배열(img_digits)에 들어 있는 3개 이미지들의 크기가 서로 다를 수 있음
- 앞에서 우리가 만든 모델에 넣어 분류 예측하기 위해서는 Input Size인 28x28 크기로 재조정

▶ # img_digits 배열에 어떠한 digit image 가 저장되어 있는지 확인해 본다.

```
for i in range(0, len(img_digits)):
    plt.subplot(1, len(img_digits), i+1)
    plt.imshow(img_digits[i], cmap='Greys', interpolation='nearest')
plt.tight_layout()
plt.show()
```



크기가 다른 이미지 3개



앞에서 우리가 만든 모델에 넣어
분류 예측하기 위해서는?

모델의 Input Size인 28×28 크기로 재조정해야 함

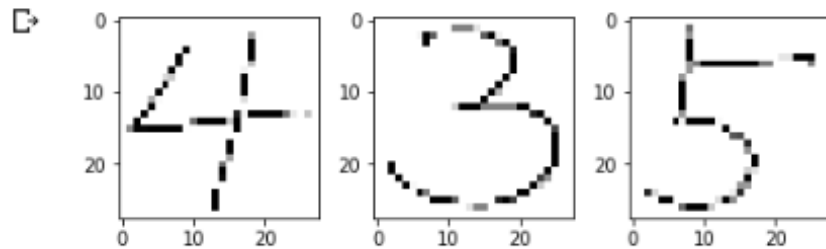


- 아래 결과에서 숫자 이미지 배열(img_digits)에 들어 있는 이미지들의 크기가 재조정 됨

```
# img_digits 배열의 크기가 서로 다른 이미지들을 모두 28X28 로 재조정해서 Display 합니다.

SZ = 28

for i in range(0, len(img_digits)):
    plt.subplot(1, len(img_digits), i+1)
    plt.imshow(cv2.resize(img_digits[i], (SZ, SZ)), cmap='Greys', interpolation='nearest')
plt.tight_layout()
plt.show()
```



이미지 크기 (28,28) 재조정

🔄 학습된 모델을 불러옴

```
1 import tensorflow as tf
2
3 # 저장해 두었던 모델을 불러옵니다.
4 model = tf.keras.models.load_model('gdrive/My Drive/CV/Digits Classify/model/digits_model.h5')
5 model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
reshape (Reshape)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 26, 26, 32)	320
conv2d_1 (Conv2D)	(None, 24, 24, 64)	18496
max_pooling2d (MaxPooling2D)	(None, 12, 12, 64)	0
dropout (Dropout)	(None, 12, 12, 64)	0
flatten_1 (Flatten)	(None, 9216)	0
dense (Dense)	(None, 128)	1179776
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1290

Total params: 1,199,882
 Trainable params: 1,199,882
 Non-trainable params: 0

■ 숫자 이미지 배열(img_digits)에 들어 있는 이미지를 모델에 넣어 분류 예측 수행

```
1  # img_digits 배열 있는 이미지들을 우리가 만든 모델에 넣어 분류 예측해 봅니다.
2
3  SZ = 28
4
5  for i in range(0, len(img_digits)):
6      plt.subplot(1, len(img_digits), i+1)
7
8      # img_digits 배열의 크기가 서로 다른 이미지들을 모두 28x28 로 재조정한다.
9      num = cv2.resize(img_digits[i], (SZ,SZ))
10
11     # 픽셀 값을 0~1 범위로 변환
12     num = num.astype('float32') / 255.
13
14     # 이미지를 Display 합니다.
15     plt.imshow(num, cmap='Greys', interpolation='nearest');
16
17     # 이미지를 우리가 만든 모델을 통해 분류 예측해 봅니다.
18     result = model.predict(np.array([num]))
19     result_number = np.argmax(result)
20     plt.title(result_number)
21     performance = [val for val in result[0]]
22     print(performance)
23 plt.tight_layout()
24 plt.show()
```

■ 숫자 이미지 배열(img_digits)에 들어 있는 이미지를 모델에 넣어 분류 예측 수행

- 실행 결과 3개의 이미지 모두 잘 분류된 것을 볼 수 있음
- 정확도 : 100%

```
1/1 [=====] - 7s 7s/step
[-23.048878, -0.755213, -8.61092, -8.808973, 14.333097, 1.6890295, -4.914475, -2.3810434, -8.059347, -5.5007377]
1/1 [=====] - 0s 18ms/step
[-20.167288, -12.058827, -1.525633, 24.027637, -18.978643, 2.8124976, -12.900267, -16.627192, -3.6454177, -11.357154]
1/1 [=====] - 0s 14ms/step
[-17.650635, -8.693256, -15.256943, 4.7381725, -10.307469, 15.189797, -4.3567514, -4.285175, -6.7587085, -3.0488234]
```

